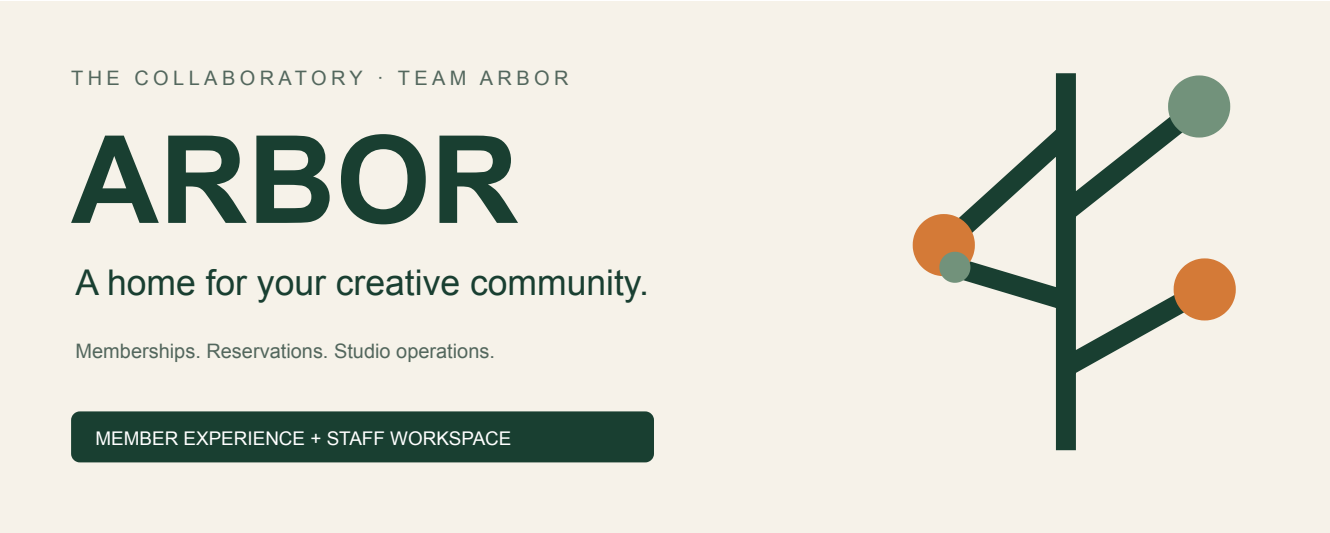
ARBOR · Sponsor setup & operations

For Steph and the next person responsible for the studio's website.



ARBOR — a home for your creative community

Read this first: you do not need AI to follow this guide. You do need access to the studio's domain, a sponsor-owned hosting account and someone authorized to approve member data and business policies. Work through one numbered section at a time. Do not invite members until the launch checklist passes.

This guide adds a **new, standalone installation**. The team's RIT/RLES server remains separate. Nothing here transfers existing records or changes the live site automatically. The guide and templates require a real server rehearsal before being treated as a completed production handoff.

---

1. Choose your route

- **Just explore the app:** ask the team for a demo, or follow Local development. Sample records are disposable.
- **Host a new sponsor-owned site:** follow sections 2–10. Recommended starting route: one **AWS Lightsail Ubuntu 24.04 LTS** server, with Docker running the app, database and HTTPS front door.
- **Use another host:** a DigitalOcean Ubuntu 24.04 LTS Droplet, or an equivalent Ubuntu VM with a stable public IPv4 address, can use the same steps beginning at section 3. The provider must allow inbound 80/443 and outbound HTTPS/email API traffic. Static-site-only hosting cannot run this database-backed application.
- **Keep using RLES:** stop here and use the team access guide. Coordinate a supported long-term owner before the school environment ends.

A reasonable **starting size**, not a load-tested capacity guarantee, is 2 CPUs, 4 GB RAM and 80 GB storage. AWS currently lists that public-IPv4 Linux bundle at **$24/month for the server**. Domain registration, snapshots, email, taxes and usage outside allowances can add costs. Confirm the price shown before buying. Lightsail pricing. Alternative provider: create a Droplet.

Keep this ownership sheet privately

Item	Write down / verify
Hosting owner	Sponsor-controlled email, billing contact, MFA and recovery method
Server	Provider, region, name, stable IP and approved monthly budget
Domain	Registrar, renewal owner and access to DNS records
Website	Intended address, for example `members.yourstudio.com`
Software	Reviewed release commit, `ARBOR_VERSION`, date and technical contact
Email	Sponsor-owned Brevo account, verified sender and reply-to inbox
Recovery	Private off-server backup location, last restore date and who can recover it

Do not put passwords, private keys, member records or backup files in this sheet if it will be shared in GitHub, a public document or an AI chat.

---

2. Understand what will run



Hosting map

The browser reaches **Caddy**, which manages HTTPS certificates. Caddy forwards requests to **ARBOR**, and ARBOR reads/writes a private **PostgreSQL** database. Brevo sends account-confirmation and configured notification email. The database is not published directly on the internet.

Important release limits

- Registration needs working confirmation email before a new user can sign in.
- Payment records can describe money received externally. They are not proof the application charged a card. Stripe integration accepts **test keys only** in this release; live keys are rejected. Keep Stripe fields blank for this setup.
- Physical NFC/door equipment needs its own keys, reader configuration, network route and supervised hardware test. A healthy website does not prove a door works.

- The forgotten-password screen is not a completed reset service. Agree a support process with the team before launch; never edit password hashes by hand.
- The team must approve policies, pricing, permissions and data before launch.

---

3. Create the server and website address

In your hosting provider's website:

1. Sign in to the sponsor-owned account and enable MFA. Set billing notifications.
2. Create a Linux server using **Ubuntu 24.04 LTS**, with a stable public IPv4 address and enough disk for the database, images, builds and backups. In Lightsail choose the OS-only Ubuntu blueprint, not a WordPress blueprint.
3. Give it a recognizable name such as `arbor-studio`. Record the region and IP.
4. In Lightsail attach a **static IP** so the address survives instance stops.
5. Set the cloud firewall: public TCP **80 and 443**; SSH **22 only from your trusted IP** (or the provider's documented browser-SSH range). Do not open **5432** or **8080**. Preserve your current SSH route while changing rules.
6. At the domain's DNS provider, add an **A record** for your chosen hostname pointing to the static IPv4 address. Do not leave an AAAA record pointing to an unrelated IPv6 server. Allow time for DNS to update.
7. Open the provider's SSH terminal for the server. Commands below run **inside that server terminal**, not in your browser address bar or your laptop terminal.

Expected result: you have a server shell and a domain pointed to its stable IP. Do not switch an existing live domain until the restored records and replacement site have been tested. Use a new test subdomain for rehearsal.

Provider references: Create a Lightsail instance, attach a static IP, firewall rules.

---

4. Install the tools and obtain the reviewed software

Install Docker on the Ubuntu server

These commands use Ubuntu's `sudo` administrator access **on the new server**. Paste one block at a time. If the OS is not Ubuntu 24.04, stop and use the official instructions for that OS instead.

```
sudo apt-get update
sudo apt-get install -y ca-certificates curl git openssl
sudo install -m 0755 -d /etc/apt/keyrings
sudo curl -fsSL https://download.docker.com/linux/ubuntu/gpg -o /etc/apt/keyrings/docker.asc
sudo chmod a+r /etc/apt/keyrings/docker.asc
```

```
. /etc/os-release
printf 'Types: deb\nURIs: https://download.docker.com/linux/ubuntu\nSuites: %s\nComponents: stable\nSigned-
        By: /etc/apt/keyrings/docker.asc\n' "$VERSION_CODENAME" | sudo tee
        /etc/apt/sources.list.d/docker.sources >/dev/null
sudo apt-get update
sudo apt-get install -y docker-ce docker-ce-cli containerd.io docker-buildx-plugin docker-compose-plugin
sudo usermod -aG docker "$USER"
```

Close that SSH session and reconnect. The Docker group is administrator-equivalent: only give trusted server operators this access. Then check:

```
docker version
docker compose version
docker run --rm hello-world
```

Expected: client/server version output, a Compose version, and “Hello from Docker!” If permission is denied, reconnect before trying again. Do not install a second Docker variant on top of an existing installation without technical help. Official procedure: Docker Engine on Ubuntu.

Get the exact reviewed release

Ask the team for the **full reviewed commit ID that includes this guide**. Do not use an old commit that predates `docs/hosting/`. A commit ID is a version label, not a password. In the following command replace `REVIEWED_COMMIT_ID` first.

```
git clone https://github.com/ISTE-500-Team-2/srdevproject.git arbor
cd arbor
git checkout REVIEWED_COMMIT_ID
git rev-parse HEAD
```

Record that last output on the ownership sheet. **Expected:** `docs/hosting/arbor.sh` exists. Do not continue with an unreviewed version just because it is newer.

---

5. Add your settings and private keys

From the `arbor` repository directory:

```
cp docs/hosting/sponsor.env.example .env.sponsor
chmod 600 .env.sponsor
nano .env.sponsor
```

In the editor, replace the example hostname and emails with your real values. `PUBLIC_HOST` is only the hostname: **no `https://` and no slash**. Set `TLS_EMAIL` to the certificate contact inbox, and `STUDIO_CONTACT_EMAIL`/phone to the approved support details. Keep `ARBOR_VERSION=handoff-01` for this first installation and record its matching Git commit. Save with **Ctrl+O, Enter**, then exit **Ctrl+X**.

Leave `EMAIL_ENABLED=false` only for the first technical health check. Leave Stripe and reader keys blank. Do not invite users until section 8 enables working email.

Generate the database password and signing key **once, on a new installation**:

```
umask 077
mkdir -p .sponsor-secrets
if [ ! -e .sponsor-secrets/db-password ]; then
  openssl rand -hex 32 > .sponsor-secrets/db-password
fi
if [ ! -e .sponsor-secrets/jwt.key ]; then
  openssl rand 32 > .sponsor-secrets/jwt.key
fi
sudo chown 1000:1000 .sponsor-secrets/db-password .sponsor-secrets/jwt.key
chmod 600 .sponsor-secrets/db-password .sponsor-secrets/jwt.key
chmod 700 .sponsor-secrets
```

The container's application user is UID 1000, so it must be able to read the mounted files. On the standard Ubuntu image the login user is normally UID 1000 too; if `chmod` reports permission denied after `chown`, use `sudo chmod 600` on those two files. Never print the key contents into support chats. **Never regenerate the database password after the database has been initialized**; changing a file does not update the password stored in PostgreSQL.

`.env.sponsor`, `.sponsor-secrets/` and `.sponsor-backups/` are excluded from Git and the image build. Keep private recovery copies outside the server as well.

---

6. Build, then choose ONE database path

Run from the repository root:

```
bash docs/hosting/arbor.sh check
bash docs/hosting/arbor.sh build
bash docs/hosting/arbor.sh start-db
```

Expected: configuration parses, image build completes, and the database becomes healthy. A build may take several minutes. A parsing result is not a deployment test.

Path A — genuinely new studio database

Choose this only when **there are no existing records to preserve**:

```
bash docs/hosting/arbor.sh initialize-new
```

Read the message, then type `CREATE NEW DATABASE` only if this is the intended empty installation. The helper refuses a database with public tables. It applies the base schema and versioned migrations, but **does not load sample users or business records**. The base DDL contains destructive statements; never run it directly over an existing database.

Path B — move or recover existing records

Do not initialize a new schema first. Have the current authorized operator provide a verified PostgreSQL 16 custom-format backup and the matching release and migration history. Inventory any separately stored files/integration keys too; a database archive is not a backup of every external system.

Transfer the archive privately into `.sponsor-backups/` on the new server. Set its permissions to owner-only. Replace the filename below:

```
bash docs/hosting/arbor.sh restore-new .sponsor-backups/verified-transfer.dump
```

Type `RESTORE NEW DATABASE` only for the intended empty database. A failed restore may leave a partial schema: **do not rerun initialization or delete volumes**. Keep the source intact and ask the operator to investigate a fresh recovery target.

Have the team verify member counts, memberships, policies, certifications and latest transactions against the source. If the reviewed release requires new migrations, approve them only after that verification:

```
bash docs/hosting/arbor.sh migrate
```

It takes an archive backup before applying migrations and asks for `APPLY REVIEWED MIGRATIONS`. Preserve the original source archive and source server until recovery acceptance is complete. Use existing administrator accounts after a restore; do not bootstrap another first administrator.

---

7. Start the website and verify HTTPS

```
bash docs/hosting/arbor.sh start
bash docs/hosting/arbor.sh status
```

Expected: `db` and `app` are healthy, and `proxy` is running. From your own computer, open `https://YOUR_HOSTNAME/api/health` and then `https://YOUR_HOSTNAME`. The health URL must return HTTP 200; the site should have a valid HTTPS certificate. The production login must **not** show working demo-login buttons.

Caddy obtains/renews certificates when DNS reaches the server and public 80/443 are reachable. A wrong hostname or blocked port prevents this. Do not disable certificate verification to hide a failure. Caddy HTTPS requirements.

---

8. Turn on email and create the first administrator

1. Sign in to the sponsor’s Brevo account. Verify an approved studio sender and authenticate the sending domain using Brevo’s current DNS instructions. Keep the domain registrar login with the sponsor.

2. Create a transactional API key. In `nano .env.sponsor`, set `BREVO_API_KEY`, `EMAIL_FROM`, `EMAIL_REPLY_TO`, `EMAIL_FROM_NAME` and the approved physical `EMAIL_POSTAL_ADDRESS`. Set **`EMAIL_ENABLED=true`**. Do not paste the key here or into a frontend file. Leave optional webhook settings blank unless the team configures the delivery-event callback separately.
3. Save, then apply the environment change:

```
bash docs/hosting/arbor.sh restart
```

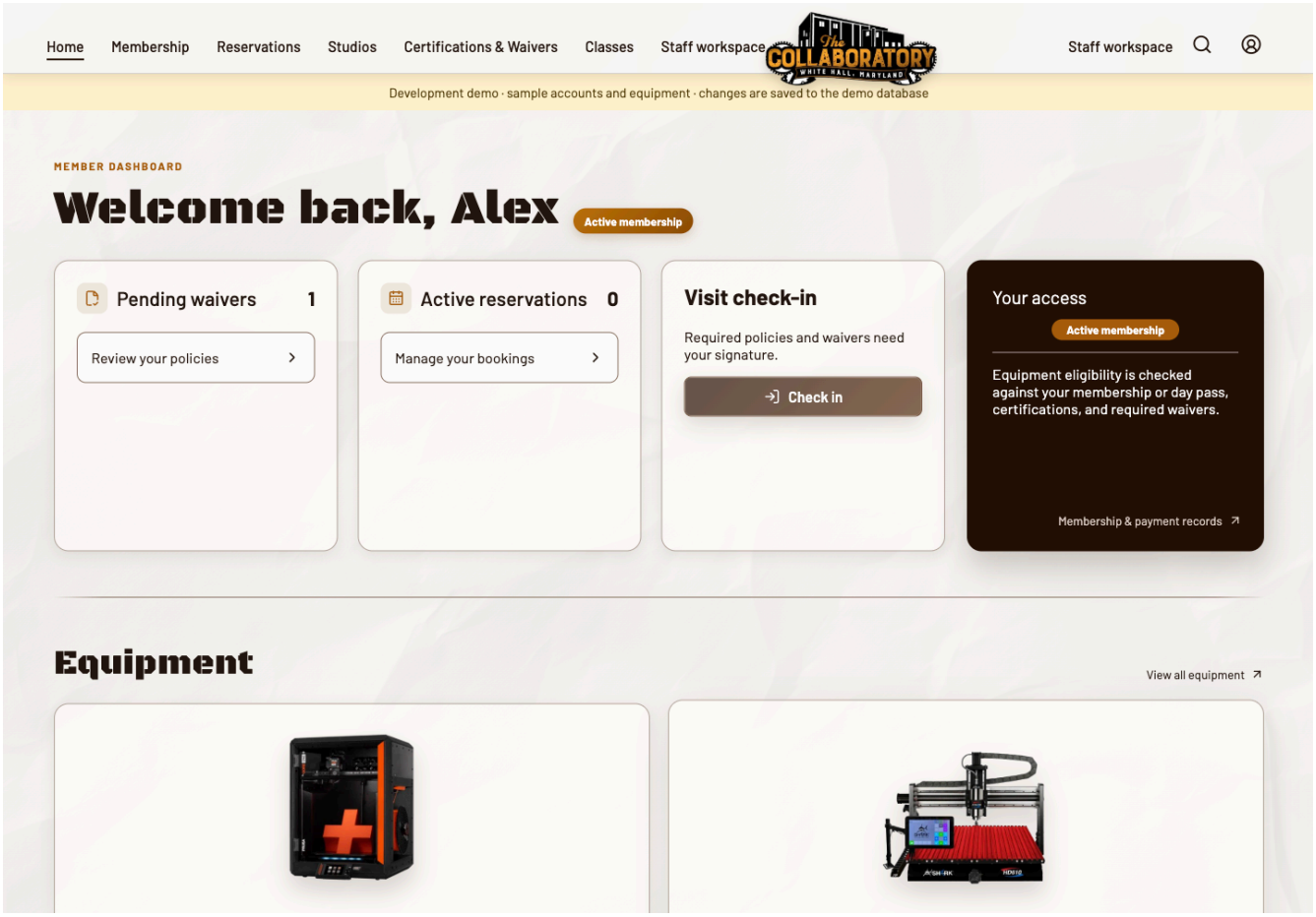
4. On the website, register **your own real owner email** and a private password. Open the confirmation email and finish confirming. If no email arrives, stop and check Brevo/logs; do not bypass confirmation in the database.
5. For a genuinely new database with **no active administrator**, run this command with that confirmed email:

```
bash docs/hosting/arbor.sh promote-owner owner@yourstudio.com
```

Type `APPROVE FIRST ADMIN`. The script only promotes an active, confirmed account when no active administrator exists, and records an audit entry. Sign out and back in, then verify you can open the Staff workspace. This is a one-time bootstrap, not the normal way to manage staff. Later, an administrator grants roles through the application. Never share the owner password.

References: Brevo sender/domain setup, domain authentication, application email guide.

9. Configure the studio and rehearse the workflows



Member dashboard in the sample-data demo

Illustration only: this screenshot is from an isolated local development demo. Your new installation starts without these synthetic accounts and business records.

In Staff workspace, configure the sponsor-approved plans, prices and policy text. Use real approval references for waivers. Have the team guide the equipment, room/studio and training setup; do not assume a new empty database includes an approved catalog. Register staff separately and grant only the intended roles.

Rehearse with dedicated test accounts before inviting members:

- Register → receive confirmation → confirm → sign in/out → sign back in.
- Create/issue a test plan and review its saved details as the member.
- Publish the approved test policy; verify a missing required waiver blocks access.
- Verify missing/revoked training prevents restricted equipment access, and approved training changes eligibility correctly.
- Create/cancel a reservation and verify another member cannot access its records.
- Record an **external test payment record**, verify the audit trail and report, then correct/void it through the supported workflow. Do not describe it as a card charge.
- Confirm ordinary members cannot access staff routes and staff cannot grant administrator access to themselves.

- Restart the services and verify records remain.

Record the operator, date, version and observed outcome. Failed steps are launch blockers, not items to silently check off.

---

10. Launch checklist

- ☐ Sponsor owns hosting, billing, domain and email; recovery access is documented.
- ☐ Reviewed release and image label recorded; cost approved.
- ☐ Valid HTTPS; only web ports public; SSH restricted; demo login disabled.
- ☐ Email confirmation succeeds from a real inbox; reply-to/contact information correct.
- ☐ Correct records restored and verified, or sponsor explicitly approved an empty start.
- ☐ Owner/staff roles, plans, waivers, equipment and training approved and tested.
- ☐ Backup copied off-server and restored into a separate empty test installation.
- ☐ External payments versus Stripe test-mode limits understood; no live billing claim.
- ☐ Hardware separately accepted before using the site to control physical access.
- ☐ Support contact and unfinished password-reset process agreed.
- ☐ Sponsor approves the final domain switch and member invitations.

Validation boundary for this documentation: screenshots show the actual local sample-data application. The documentation review and local checks do not prove an AWS/DigitalOcean deployment, real DNS/certificate issuance, Brevo delivery, production data transfer, live payments or physical reader operation. Record an actual server rehearsal before calling this hosting path production-accepted.

---

11. Backups, recovery and routine care

After setup, before changes, and on the agreed daily schedule

```
bash docs/hosting/arbor.sh backup
```

The helper creates a private custom-format archive in `.sponsor-backups/`, checks that it is nonempty and that PostgreSQL can read its index. **That is not a restore test.** Copy it to a private, encrypted, off-server location, with sponsor-approved retention. Restrict access because archives contain member data. The command is manual: this template **does not schedule daily backups for you**. Assign an owner or have the technical contact configure and verify monitored automation.

Also keep private recovery copies of `.env.sponsor`, both secret files, the reviewed source commit and any external file/integration inventory. Caddy certificate state is in a persistent volume and can normally be reissued; losing application signing keys invalidates sessions. Losing the database volume loses records without recovery.

Lightsail automatic snapshots are an additional layer, not the only database backup. AWS retains the seven most recent daily automatic snapshots; they incur charges. Enable them in the provider console and check the bill. Snapshot setup, snapshot FAQ.

Prove you can recover

Use a **separate new server, hostname and empty database**. Follow sections 3–6, choosing **restore-new**, with the saved release/settings and private keys. Keep email and hardware integrations disabled during rehearsal so a recovery test does not send duplicate messages or operate real equipment. Review records with the technical contact before reconnecting integrations. Record the archive filename, recovered latest transaction, operator and date. Never rehearse by overwriting the only working database. Agree recovery-time and acceptable-data-loss targets with the studio rather than assuming daily backup is sufficient.

Monthly checks

Confirm backups/restores, billing/domain renewals, disk capacity, provider security updates, HTTPS and confirmation-email delivery. `df -h` shows disk space; the helper's `status` and `logs` commands show service state. Logs can contain identifiers: redact member details before sharing. Apply OS reboots in an agreed maintenance window.

---

12. Updates and rollback

Do not use an automatic `git pull` as a production update policy. Obtain a reviewed release and its database compatibility/rollback notes first.

1. Arrange a maintenance window. Record the current commit, image label and health.
2. Take a backup, copy it off-server and confirm a known-good restore exists.
3. Check out the reviewed new commit. Keep the private environment and secret files. Change `ARBOR_VERSION` to a new unique label, then run `check` and `build`.
4. If the release requires schema changes, review them and use `migrate` with the explicit confirmation. Never replace or edit already-applied migrations.
5. Run `restart`, verify health/email and rehearse the critical workflows.
6. If the update fails, stop member writes and contact the technical owner. An older image is only safe when it supports the current database schema. Restoring a database backup loses writes made after that backup: agree the recovery target and use a new empty recovery database before switching service.

Never use `docker compose down -v`, delete database volumes, run demo seed scripts, or run the legacy create scripts as troubleshooting. Keep the previous image until acceptance. The current RLES GitHub release workflow is separate from this manual sponsor setup; configuring its secrets is not required by this guide.

13. If something goes wrong

Symptom	Check / safe next step
`docker: command not found`	Finish Docker installation on the server, not your laptop.
Docker permission denied	Reconnect SSH after group change; confirm the intended trusted user.
Configuration says hostname/version missing	Edit `.env.sponsor` in the repository root; replace placeholders.
Signing key cannot be read	Check file existence, UID 1000 ownership and 600 permissions; do not regenerate it.
Database authentication fails after editing a key	Restore the original matching password file; ask the operator before rotating database credentials.
Database already has tables	The empty-database guard worked. Stop; identify whether this is existing or partially restored data.
Build runs out of memory/disk	Check server size and `df -h`; ask before resizing. Do not delete database storage to make space.
Browser cannot reach site / certificate fails	Check A/AAAA records, stable IP, public 80/443 and proxy logs.
HTTP 502 / app unhealthy	Run `status` and `logs`; inspect database/key/migration errors. Do not reseed.
Signup email missing	Confirm `EMAIL_ENABLED=true`, valid Brevo key, authenticated sender, provider activity and spam folder.
Owner promotion refused	Confirm email first. If an active admin already exists, use that admin's supported role-management flow.
UI missing catalog or plans	Empty installations have no demo business records. Complete approved studio configuration.
Hardware or real card charging unavailable	These require separate acceptance/work; do not enable demo access or live keys as a workaround.

Useful commands, from the repository root:

```
bash docs/hosting/arbor.sh status
bash docs/hosting/arbor.sh logs
```

Send the technical contact the release commit, guide step, exact error with secrets and personal details removed, and what changed immediately before the failure. Do not share `.env.sponsor`, key files or database archives in a public issue.

14. Optional AI assistance

AI is a second set of eyes, not the only copy of the instructions. You can ask:

> I’m following ARBOR Sponsor Guide section 7 on a new Ubuntu 24.04 server. The domain points to the server, but HTTPS fails. Here is the error with secrets and personal data removed: [error]. Explain the next read-only check in plain English and what successful output looks like. Do not delete data or change keys.

> Explain this guide step before I run it. Tell me where the command runs, what it changes, and how I can verify it. Do not ask for passwords or backups.

Keep account ownership, billing approval, DNS changes, data restore choices and launch acceptance with a human. If an assistant suggests removing a volume, bypassing email confirmation or enabling production demo login, stop and consult the technical owner. Share only the minimum sanitized error necessary.

Maintainers: Markdown is the editable source. Keep the printable PDF, images, Compose template and helper commands in sync when changing this guide. The source and provider references were reviewed October 16, 2026; provider screens and prices can change. For code-level integration, see MVC, Team handoff, JWT setup and requirements evidence.